Python Threading Tutorial

Contents

Python Threading Tutorial 1
Video 1
Threading 1
Process 2
Time Slicing 2
Tutorial 1: Without Threading 3
Tutorial 2: Threading 4
Tutorial 3: Threading with Join 5
Tutorial 5: Two Concurrent Threads 8
Tutorial 5: Daemon Threads and Tkinter 8
Assignment Submission 13

Time required: 60 minutes

Video

This is a 3 hour class that goes into this subject very deeply. Watch this if you want to take a deep dive into threading.

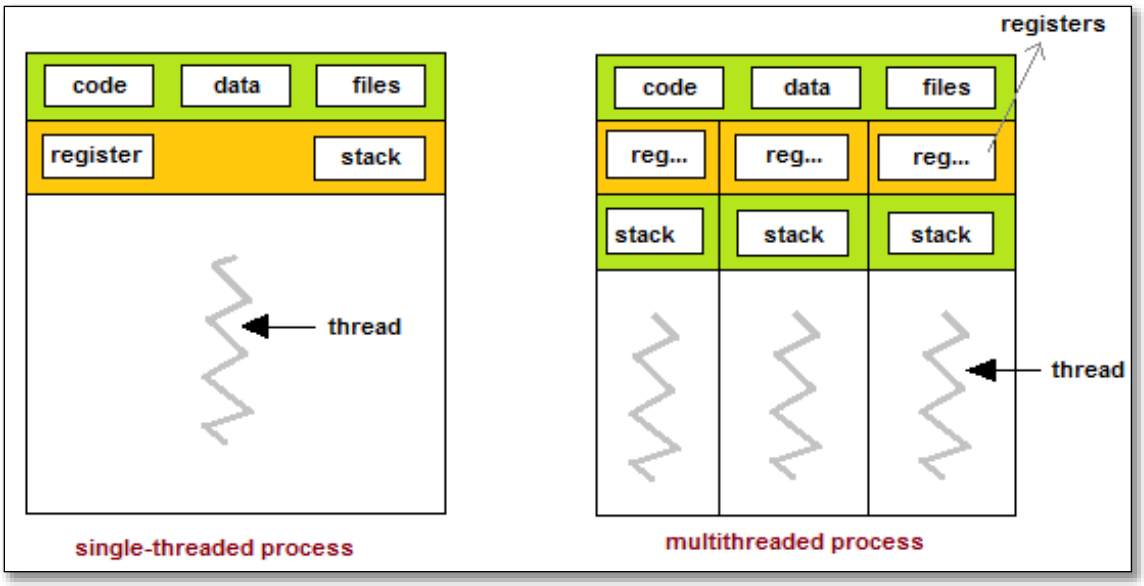
Asynchronous Python Mastery: Threads, processes, and Async Development

Threading

Threading is built into Python. Threading in Python is used to run multiple threads (tasks, function calls) at the same time. Python threads are used in cases where the execution of a task involves some waiting. One example would be interaction with a service hosted on another computer, such as a webserver. Threading allows python to execute other code while waiting.

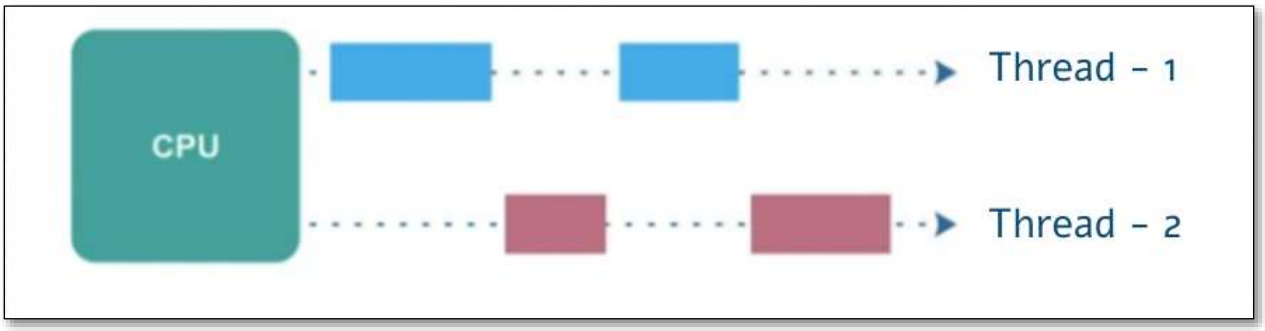
Process

A process has at least one thread and some resources set aside by the operating system. This could be a hard drive, processor cores, network, etc. The resources change as the process's needs change.



Time Slicing

Your computer can have hundreds of processes running at the same time. Time slicing or scheduling allows these threads to share process time. Time slicing is not free. The OS has to save the current status of the thread so that when it comes back to the core, it doesn't start from the beginning. There is some overhead in switching from one thread to another.



Tutorial 1: Without Threading

All program runs on at least one single thread. All code executes nose to tail, one after the other. Functions must run one after the other. These are called blocking calls. Nothing else can happen while the function is running.

- function1()
- function2()
- function3()

This program shows an example of functions running in a single thread.

```python
"""
    Filename: threading_example.py
"""
import threading


# ---------------------- TEST FUNCTIONS -----------------------------------#
def function1():
    for i in range(5):
        print("ONE ")


def function2():
    for i in range(5):
        print("TWO ")


def function3():
    for i in range(5):
        print("THREE ")


def main():
    # ------------------- NORMAL LINEAR FUNCTIONS --------------------------#
    # If we call these functions, the first function call
    # MUST complete before the next, they are executed linearly
    function1()
    function2()
    function3()


# If a standalone program, call the main function
# Else, use as a module
if __name__ == "__main__":
    main()
```

Example run:

```
ONE
ONE
ONE
ONE
ONE
TWO
TWO
TWO
TWO
TWO
THREE
THREE
THREE
THREE
THREE
```

Tutorial 2: Threading

Threading allows us to speed up programs by executing multiple tasks at the SAME time.

- Each task will run on its own thread
- Each thread can run simultaneously and share data with each other
- Every thread you start must do SOMETHING
- Threads will finish at different times. The OS task scheduler runs them when it has time.

We will define 3 different functions, one for each thread. Our threads will then target these functions. When we start the threads, the target functions will be run.

Modify the existing code as shown.

```python
def main():
    # ------------------- NORMAL LINEAR FUNCTIONS -------------------------#
    # If we call these functions, the first function call
    # MUST complete before the next, they are executed linearly
    # function1()
    # function2()
    # function3()

    # ---------------------- THREADED FUNCTIONS ----------------------------#
    # We can execute these functions concurrently using threads!
    # We must have a target for a thread.
    t1 = threading.Thread(target=function1)
    t2 = threading.Thread(target=function2)
    t3 = threading.Thread(target=function3)

    t1.start()
    t2.start()
    t3.start()
```

Example run (Each run may be different):

```
ONE
TWO
TWO
ONE
TWO
TWO
ONE
ONE
ONE
TWO
THREE
THREE
THREE
THREE
THREE
```

Tutorial 3: Threading with Join

Modify the existing code to add a print statement. Notice that it executes right away on the program thread.

```python
def main():
    # ------------------- NORMAL LINEAR FUNCTIONS -------------------------#
    # If we call these functions, the first function call
    # MUST complete before the next, they are executed linearly
    # function1()
    # function2()
    # function3()

    # ---------------------- THREADED FUNCTIONS ----------------------------#
    # We can execute these functions concurrently using threads!
    # We must have a target for a thread.
    t1 = threading.Thread(target=function1)
    t2 = threading.Thread(target=function2)
    t3 = threading.Thread(target=function3)

    t1.start()
    t2.start()
    t3.start()

    # This pauses the main program until the thread is complete
    # t1.join()
    # t2.join()
    # t3.join()
    print("Threading rules!")
```

Example run (Each run may be different):

```
ONE
ONE
ONE
ONE
ONE
TWO
THREE
THREE
THREE
THREE
THREE
TWO
TWO
TWO
TWO
Threading rules!
```

Uncomment the **.join()** methods. The .join() method releases the thread and allows the program to "join" the main thread.

This is an example run with join. All three threads must complete before the next command is run.

Example run (Each run may be different):

```
ONE
ONE
TWO
TWO
ONE
ONE
ONE
Threading rules!
THREE
THREE
TWO
TWO
TWO
THREE
THREE
THREE
```

Tutorial 5: Two Concurrent Threads

```python
# Import the threading module to create and manage threads
from threading import Thread
# Import the time module to use the sleep function
from time import sleep


def print_numbers():
    """Function to print numbers from 1 to 5 with a 1 second delay"""
    for i in range(1, 6):
        print(f"Number: {i}")  # Print the current number
        # Pause for 1 second before printing the next number
        sleep(1)


def print_letters():
    """Function to print letters A to E with a 1 second delay"""
    for letter in 'ABCDE':
        print(f"Letter: {letter}")  # Print the current letter
        # Pause for 1 second before printing the next letter
        sleep(1)


# Create a thread for each function
thread1 = Thread(target=print_numbers)
thread2 = Thread(target=print_letters)

# Start the threads, this will run the functions concurrently
thread1.start()
thread2.start()

# Wait for both threads to finish before exiting the program
thread1.join()
thread2.join()

# The main program will continue to run while the threads are running
# The threads will run concurrently, they will both be active at the same time
```

Tutorial 5: Daemon Threads and Tkinter

A daemon is a background service. If you are running a function or method in a separate thread that you want to keep going until the program stops, setting daemon to True will stop the thread when the program ends.

In a Tkinter application, the main event loop (mainloop()) is responsible for handling all GUI updates and user interactions. If a long-running task is executed within this main thread, it will block the event loop, causing the GUI to become unresponsive until the task completes.

Threading helps by allowing long-running tasks to run in separate threads, freeing up the main thread to continue handling GUI updates and user interactions. This ensures that the GUI remains responsive even while background tasks are being executed.

In the provided code, the **background_task** method runs in a separate thread created by the **start_thread** method. This allows the label to be updated with the current timestamp every second without blocking the main event loop, keeping the GUI responsive.

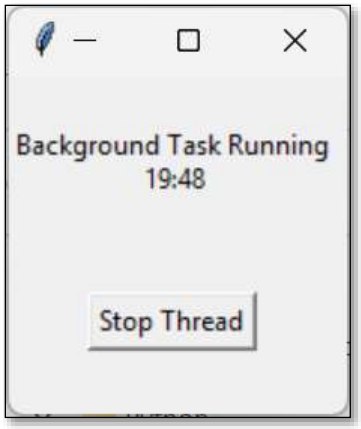
This program can be used as a template for any Python GUI program to run multiple threads.

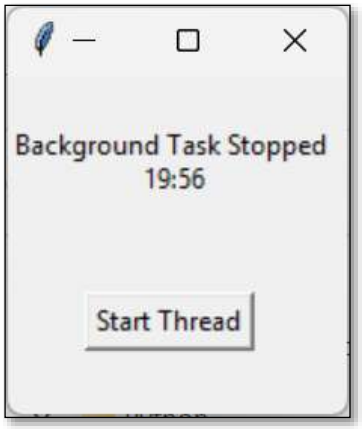
```python
import tkinter as tk
from threading import Thread, Event
from time import localtime, sleep


class ThreadingApp:
    def __init__(self):
        # Create the main window of the application
        self.root = tk.Tk()
        # Set the title of the window
        self.root.title("Threading with Tkinter")
        # Set the size of the window
        self.root.geometry("150x150")
        # Call the method to set up the GUI elements
        self.setup_gui()

        # Event to signal the thread to stop
        self.stop_event = Event()

        # Boolean flag to track the thread state
        self.is_running = False

        # Handle window close event
        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)
        self.root.mainloop()

# ---------------------------- SETUP GUI ---------------------------------- #
    def setup_gui(self):
        # Create and grid label widget to display text
        self.lbl_display = tk.Label(self.root, text="Background Task Stopped")
        self.lbl_display.grid(row=0, column=0, columnspan=2, pady=20)

        # Create and grid a button that will start/stop the thread when clicked
        self.toggle_button = tk.Button(
            self.root,                    # Parent widget is the main window
            text="Start Thread",          # Text shown on the button
            command=self.toggle_thread    # Method called when button is clicked
        )
        self.toggle_button.grid(
            row=1, column=0, columnspan=2, pady=20, padx=10)
```

```python
# ---------------------------- BACKGROUND TASK ---------------------------- #
    def background_task(self):
        """This method runs in a separate thread
           and performs a background task"""
        # While the stop event is not set, keep running the task
        while not self.stop_event.is_set():
            # Update the label text with the current timestamp
            self.get_local_time()
            self.lbl_display.config(
                text=f"Background Task Running\n {self.now}"
            )

            # Pause for 1 second before the next update
            sleep(1)

# ---------------------------- TOGGLE THREAD ---------------------------- #
    def toggle_thread(self):
        if self.is_running:
            self.stop_thread()
        else:
            self.start_thread()

# ---------------------------- START THREAD ---------------------------- #
    def start_thread(self):
        # Clear the stop event before starting the thread
        self.stop_event.clear()
        # Create a new thread object
        self.thread = Thread(
            target=self.background_task,  # Function to run in the thread
            daemon=True                   # Thread stops when program ends
        )
        # Start the thread
        self.thread.start()

        # Update the button text
        self.toggle_button.config(text="Stop Thread")

        # Update the flag value
        self.is_running = True
```

```python
# ---------------------------- STOP THREAD ------------------------------- #
    def stop_thread(self):
        # Set the stop event to signal the thread to stop
        # The thread will stop after the next iteration
        self.stop_event.set()

        # Update the button text and flag
        self.toggle_button.config(text="Start Thread")

        self.get_local_time()
        self.lbl_display.config(text=f"Background Task Stopped\n {self.now}")

        # Update the flag
        self.is_running = False

# ---------------------------- GET LOCAL TIME ---------------------------- #
    def get_local_time(self):
        """Get the current local time and format it"""
        now = localtime()
        self.now = f"{now.tm_min:02d}:{now.tm_sec:02d}"

# ---------------------------- ON CLOSING -------------------------------- #
    def on_closing(self):
        # Stop the thread when closing the window
        self.stop_thread()
        # Destroy the window
        self.root.destroy()


def main():
    app = ThreadingApp()


if __name__ == "__main__":
    main()
```

Example run:





Assignment Submission

1. Attach all program files.
2. Attach a screenshot of each successful program run.
3. Submit the assignment in Blackboard.